Agenda

- NodeJs
- BuiltIn and User Defined Modules
- Promise
- Event Loop
- http server
- ~~Express~~

CJS vs ESM

- CommonJS (CJS) and ECMAScript Modules (ESM) are two different module systems for JavaScript, each with a distinct history, syntax, and set of features.
- CJS was developed for the server-side environment of Node.js before JavaScript had a native module system, while ESM was added to the official JavaScript language standard (ES6) for both browser and server environments.

How to use ESM in Node.js

- we have two primary options for working with ESM in Node.js

1. Use .mjs files. This is the most explicit method. All files ending in .mjs will be automatically parsed and run as ECMAScript Modules.
2. U se type: "module". For projects that want to use ESM throughout, you can add "type": "module" to your package.json. All .js files will then be treated as ESM. This also allows you to explicitly use CJS for a file by naming it with the .cjs extension.

JavaScript as SingleThreaded

- JavaScript is single-threaded, but it has the unique ability to handle asynchronous tasks in a non-blocking manner.
- This apparent contradiction is resolved by a key architectural feature of the JavaScript runtime environment: the Event Loop.
- To be single-threaded means that the JavaScript engine has only one call stack and can execute only one piece of code at a time.
- The engine processes code sequentially, line by line, on this single thread.
- For most code, this is fine. If a function is called, it is added to the call stack and executed. If that function calls another function, the new function is added on top.
- However, this could be a problem for long-running operations. If the single thread gets stuck waiting for a database query, network request, or a file to load, the entire application would freeze and become unresponsive.

How asynchronous programming works in JS

- Asynchronous programming is the technique that allows JavaScript to perform these long-running tasks without blocking the main execution thread.
- It is a way to handle operations concurrently, not simultaneously.

SUNBEAM INFOTECH

- This is made possible by the JavaScript runtime environment (like a web browser or Node.js), which consists of several components:

1. Call Stack:

- As mentioned, this is where synchronous JavaScript code is executed. It works on a Last-In, First-Out (LIFO) principle.

2. Web APIs / Background APIs:

- These are features of the runtime environment, not the JavaScript engine itself. When the JavaScript engine encounters an asynchronous task (like setTimeout, a fetch request, or an event listener), it delegates the task to the appropriate API.
- The API runs the operation in the background, freeing up the main thread to continue executing other code.

3. Callback Queue:

- After a background task is completed by a Web API, its associated callback function is placed in a queue, where it waits to be executed.

4. Microtask Queue:

- A separate queue for high-priority asynchronous tasks, such as those from Promises and async/await.
- The Event Loop prioritizes the Microtask Queue over the Callback Queue.

5. Event Loop:

- This is the core mechanism that connects everything.
- It constantly monitors the call stack. When the stack is empty, it checks the Microtask Queue first, then the Callback Queue.
- If it finds any pending callbacks, it moves the first one onto the call stack for execution.

Event Loop & libuv

- The Concept (Event Loop): The idea of a single main thread that delegates long-running tasks and then processes callbacks from a queue after the stack is clear. This pattern allows for high concurrency without the overhead of creating a new thread for every request.
- The Implementation (libuv): The actual code that makes the event loop possible in Node.js. Libuv abstracts the underlying complexities of different operating systems (like using epoll on Linux, kqueue on macOS, and IOCP on Windows) to provide a consistent event-driven API.
- The Event Loop is the mechanism in JavaScript (and Node.js) that allows asynchronous and non-blocking execution.
- It runs on the main thread
- It Manages callbacks and events
- Uses queues to execute things when they’re ready
- Even though JavaScript is single-threaded, Node.js can handle many tasks at once because of the event loop + callback queue + background workers(libuv).
- libuv is the C++ library that powers Node.js’s I/O.

SUNBEAM INFOTECH

- It Manages the event loop
- Handles file system, DNS, timers, sockets
- Uses a thread pool for non-blocking execution

Example with code and flow visualization

```
console.log("Start");

setTimeout(() => {
  console.log("From Macrotask");
}, 0);

Promise.resolve().then(() => {
  console.log("From Microtask");
});

console.log("End");
```

- Step 1: Start Execution
    - console.log("Start") is pushed onto the Call Stack.
    - It is executed, and "Start" is printed to the console. The stack is now empty.
- Step 2: Offload setTimeout
    - setTimeout() is pushed onto the Call Stack. It is a Web API, so the timer is offloaded to the Background APIs.
    - The setTimeout function is popped off the Call Stack. The main thread is not blocked.
- Step 3: Offload Promise
    - Promise.resolve() is pushed to the Call Stack, which immediately resolves.
    - Its .then() callback is a microtask.
    - The microtask is placed into the Microtask Queue.
- Step 4: Continue Synchronous Code
    - console.log("End") is pushed onto the Call Stack.
    - It is executed, and "End" is printed to the console. The stack is now empty.
- Step 5: Event Loop Activation (Microtasks)
    - The Event Loop sees the Call Stack is empty.
    - It checks the Microtask Queue, finds the promise callback, and pushes it onto the Call Stack.
    - The callback executes, and "From Microtask" is printed. The Call Stack is empty again.
- Step 6: Event Loop Activation (Macrotasks)

SUNBEAM INFOTECH

    - The Event Loop checks the Microtask Queue again (it's empty).
    - It then checks the Macrotask Queue, finds the setTimeout callback, and pushes it onto the Call Stack.
    - The callback executes, and "From Macrotask" is printed. The Call Stack is empty.

```
Start
End
From Microtask
From Macrotask
```

- This demonstrates how microtasks (from the Promise) are prioritized over macrotasks (from setTimeout), even though the setTimeout had a 0ms delay.

REST (Representational State Transfer) Services

- It is a design pattern
- A request is sent and response is received
- response is always in the form of JSON
- request consists of 2 parts atleast
    1. HTTP method (get,post,put,delete) operation
        - get used to select
        - post used to submit
        - put used for update/edit
        - delete used to remove
    2. path

http server

- An HTTP server is a program/application that listens for incoming HTTP requests (like from a browser) and sends back HTTP responses (like HTML, JSON, etc.).
- In Node.js, we can build an HTTP server using the built-in http module

SUNBEAM INFOTECH

```
const http = require('http');

// Create server
const server = http.createServer((req, res) => {
  // Set headers
  res.writeHead(200, { 'Content-Type': 'text/plain' });

  // Send response
  res.end('Hello from Node.js HTTP server!');
});

// Start listening on port 3000
server.listen(3000,'localhost', () => {
  console.log('Server is running at port 3000');
});
```

- `http.createServer()` Creates the server object
- `req` Represents the incoming request (URL, headers, etc.)
- `res` Used to send back data (HTML, JSON, etc.)
- `.listen()` Starts the server on a specific port

To handle the routes in http server

```
const server = http.createServer((req, res) => {
  if (req.url === "/" && req.method === "GET") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>Home Page</h1>");
  } else if (req.url === "/about") {
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end("<h1>About Page</h1>");
  } else {
    res.writeHead(404);
    res.end("Page not found");
  }
});
```

Routing

- Routing refers to determining how an application responds to a client request to a particular endpoint, which is a URI (or path) and a specific HTTP request method (GET, POST, and so on).
- It maps a particular URL or URL pattern to a specific function that handles the request.
- Routing is like a map that helps the server know how to respond to different types of requests made by a user or client.
- For example, when a client sends a request like:
  - GET /home
  - POST /user

SUNBEAM INFOTECH

- The routing system determines which function or "route handler" should be called to respond to that request.

nodemon

- It is a developer tool that automatically restarts your Node.js app when file changes in the directory are detected.
- saves you from having to manually stop and restart your server every time you edit code.
- In short it monitors your files and restarts server automatically
- install it globally using below command

```
npm install -g nodemon
```

- to Use It

```
nodemon server.js
```

packages managers

1. npm

- Node Package Manager: World's largest software registry for JavaScript packages.

```
# by default gets installed with nodejs

# install express
> npm install express

# install all packages mentioned in package.json
> npm install
```

- Key Functions:
  - Install/uninstall packages
  - Manage dependencies
  - Run project scripts
  - Publish packages

Command	Effect	Saves to
npm install package	Local install	node_modules
npm install -g package	Global install	System-wide

- npm alternatives pnpm: Disk-efficient yarn: Faster, deterministic installs

SUNBEAM INFOTECH

2. pnpm

- pnpm is up to 2x faster than npm
- Files inside node_modules are cloned or hard linked from a single content-addressable storage

```
# install pnpm
> npm install -g pnpm

# install express
> pnpm install express

# install all packages mentioned in package.json
> pnpm install
```

3. yarn

- It provides faster installation speeds due to its ability to install packages in parallel and its utilization of a local cache for previously downloaded dependencies.
- ensures consistent installations across different environments by using a yarn.lock file.

```
# install yarn
> npm install -g yarn

# install express
> yarn add express

# install all packages mentioned in package.json
> yarn
```

SUNBEAM INFOTECH